



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-42507 Go net/textproto error-message injection via unescaped input

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-07
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-42507 Go net/textproto error-message injection via unescaped input - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-07 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-42507 Go net/textproto error-message injection via unescaped input

1. Executive Summary

CVE-2026-42507 is a medium-severity input-handling flaw in Go's standard-library `net/textproto` package. Go's vulnerability database, OSV record, NVD entry, and the upstream Go issue all describe the same core problem: when certain `net/textproto` parsing paths return an error, attacker-controlled input can be embedded in that error string without escaping.

The practical risk is log or output injection rather than direct remote code execution. Downstream applications that print, persist, alert on, or parse those error strings can end up recording misleading attacker-controlled text. Exposure is most relevant to Go services that accept untrusted text-based protocol input and either log raw parser errors or propagate them into monitoring, console output, or other security-relevant telemetry.

2. Vulnerability Metadata

Field	Value
CVE	<code>`CVE-2026-42507`</code>
Go advisory ID	<code>`GO-2026-5039`</code>
Product / component	Go standard library <code>net/textproto`</code>
Vulnerable behavior	error paths include unescaped input as part of returned error text
Primary weakness framing	improper output neutralization / log injection style behavior; reviewed public snippets did not expose a stable CWE assignment during this task
CVSS v3.1	<code>`5.3 / MEDIUM`</code>
Affected version branches	before <code>`Go 1.25.11`</code> , and <code>`Go 1.26.0`</code> through before <code>`Go 1.26.4`</code>
Fixed versions	<code>`Go 1.25.11`</code> and <code>`Go 1.26.4`</code>
Public disclosure date in Go / NVD sources	<code>`2026-06-02`</code>
Upstream tracking	Go issue <code>`#79346`</code>

3. Source Review & Confidence

Source	Contribution	Confidence
Reviewed source	CVE-2026-42507 is listed by MSRC and was the originating feed item for this intake review.	Useful as the intake and publication reference, but not the richest technical source.
Reviewed source	Go vulnerability database identifies <code>`GO-2026-5039`</code> , <code>`CVE-2026-42507`</code> , and <code>net/textproto`</code> as the affected package; issu	Primary vendor-maintained vulnerability source for Go.
Reviewed source	Public OSV summary/search snippet exposes affected/fixed version boundaries (<code>`1.25.11`</code> , <code>`1.26.4`</code>) and affected imports/s	Strong corroborating source for version ranges and package scope.
Reviewed source	Upstream issue states that arbitrary input	Strong source for exploitation framing and

	can appear in `net/textproto` errors and may affect downstream consumers such	downstream impact context.
Reviewed source	Release history states that Go `1.25.11` and `1.26.4` include security fixes for `net/textproto`.	Primary release-validation source.
Reviewed source	NVD records the CVE as published on 2026-06-02 with Go Project as the source.	Useful for publication status and severity/corroboration, but less detailed than Go-native sources.
Reviewed source	Debian tracks the same issue and links the upstream issue and fix commit lineage, corroborating package impact and patch	Good downstream corroboration.

Confidence is high on the core finding because multiple primary and near-primary sources align: Confidence is lower on downstream exploit chaining because the reviewed public material does not document in-the-wild exploitation, CISA KEV inclusion, or a broadly weaponized proof of concept. The strongest supportable claim is that this is a telemetry and integrity problem for applications that surface parser errors unsafely.

4. Vulnerability Details

`net/textproto` underpins parsing logic for text-based protocols in Go. The vulnerability arises when error-generation paths incorporate the offending input directly into the returned error string without escaping or neutralizing control content first.

That means a malicious peer can potentially supply crafted text containing newline characters or other deceptive content so that, when the application logs or displays the resulting error, the record no longer faithfully reflects a single clean parser failure. Depending on how the application handles those errors, an attacker may be able to:

- inject forged-looking lines into logs or console output
- confuse alert triage or incident review with misleading parser errors
- poison downstream automation that scrapes or pattern-matches raw error text
- obscure the original triggering input or the true parsing failure context

The reviewed sources do not support a claim that the bug alone yields code execution. The realistic impact is corruption or deception of operational telemetry and analyst interpretation.

5. Attack Scenarios

1. A Go service accepts attacker-controlled SMTP, HTTP-adjacent, or other text-protocol input that eventually reaches `net/textproto` parsing logic. Malicious content is reflected into an error and written verbatim into structured or unstructured logs.
2. A backend parser error is forwarded into SIEM enrichment, customer-visible troubleshooting output, or operator dashboards. Injected content causes responders to misread the event history or chase false indicators.
3. A security-sensitive automation workflow keys off exact parser error text. Crafted input alters logged output enough to break correlation, hide repeated failures, or create misleading alert narratives.

The attacker still needs a delivery path into a vulnerable Go application and a code path that logs or displays the resulting error. Risk is therefore situational but real for internet-facing services and protocol parsers that treat error strings as trustworthy telemetry.

6. Threat Landscape & Exploitation Status

- Public disclosure exists across MSRC, Go's vulnerability database, OSV, NVD, Debian, and community aggregation sites.
- The upstream issue indicates the Go maintainers expected relatively limited impact, which is consistent with a medium-severity telemetry-integrity flaw rather than a broad memory-corruption issue.
- No evidence was collected during this review showing CISA KEV inclusion or confirmed in-the-wild exploitation.

- Exploitation appears most plausible in environments where parser errors are heavily trusted for security operations, auditability, or automated response.

7. Affected Products and Exposure

The most defensible exposure statement is that affected products are Go applications built against vulnerable maintenance releases and using ``net/textproto`` parsing paths with untrusted remote input.

Higher-risk patterns:

- Internet-facing Go services that parse text-based protocols and log raw parser failures.
- Internal gateway, mail, proxy, or middleware components whose logs feed SIEM, ticketing, or alert-enrichment pipelines.
- Multi-tenant services where attacker-controlled input can deliberately generate high volumes of parser errors for deception or noise.
- Products that expose raw backend parser errors to customers, operators, or support tooling.

Lower-risk patterns:

- Applications already rebuilt or upgraded to Go ``1.25.11`` or ``1.26.4`` (or later maintained releases).
- Services that sanitize, normalize, or suppress raw parser error text before logging or display.
- Environments where parser errors are not exposed to users and have little operational decision value.

8. Mitigation and Workarounds

Priority actions:

4. Upgrade Go toolchains and rebuild affected applications to ``1.25.11``, ``1.26.4``, or later maintained releases.
5. Review whether applications log raw ``net/textproto`` parser errors directly; sanitize or structure those messages before they enter logs, dashboards, or alerts.
6. Prefer log fields that separate trusted context from raw peer input so attacker-controlled text cannot masquerade as a system-generated event.
7. Review monitoring rules or automation that depend on exact error-string matching and harden them against newline or content-injection edge cases.
8. Where patching is delayed, reduce unnecessary exposure of raw parser errors to end users or external tenants.

9. Detection Engineering

Detection is primarily about vulnerable-version discovery and telemetry hygiene:

Signal	Telemetry Source	Analytic Goal
Inventory Go versions used to build internet-facing services, protocol handlers, mail components, and middleware that may use <code>`net/textproto`</code> .	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Hunt application logs for parser-error entries containing embedded line breaks, suspicious prefixes, or attacker-like payload fragments.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Review logging middleware for cases where raw error strings are emitted into JSON, CSV, syslog, or line-oriented logs without normalization.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Flag services still pinned below <code>`1.25.11`</code> on the 1.25 branch or below <code>`1.26.4`</code> on the	Asset inventory / service / security	Identify vulnerable systems, exposure, or

1.26 branch.	telemetry	suspicious follow-on activity
Validate that alerting and SIEM parsers treat multi-line or malformed error content safely.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-42507 Go net/textproto error-message injection via unescaped input
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype", "sbom")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which internet-facing Go services were built with versions earlier than `1.25.11` or in the vulnerable `1.26.0`-`1.26.3` range?
- Which services ingest text-based protocol content and log parser errors directly?
- Do recent logs show malformed or multi-line parser errors that appear to include client-supplied text?
- Are there dashboards, support portals, or customer-visible diagnostics that surface raw parser failures from Go services?
- Do any detection or response playbooks rely on exact parser error strings that could be spoofed or polluted?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with high confidence for this intake review. The vulnerability is real and operationally relevant, but the reviewed public material describes a parser-error injection issue rather than a documented intrusion chain or adversary procedure that would justify a precise ATT&CK mapping without over-claiming.

12. Validation / Lab Testing

This intake review validated the report against public advisory and release material rather than performing a fresh exploit reproduction. Analyst validation steps completed:

- Reviewed the MSRC Update Guide listing that triggered the intake.

- Reviewed Go vulnerability database and OSV public summaries for advisory scope, symbol names, and fixed-version boundaries.
- Reviewed the upstream Go issue summary for downstream impact context affecting `net/http` and `net/smtp` consumers.
- Reviewed Go release-history entries showing the security fixes landed in `1.25.11` and `1.26.4` on 2026-06-02.
- Reviewed NVD publication metadata and Debian tracker corroboration.

No Lost Boys Cyber proof-of-concept execution was performed as part of this draft review.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	Platform / Application Owners	Patch affected Go toolchains to `1.25.11`, `1.26.4`, or later and rebuild exposed services.
High	SOC / Detection Engineering	Treat this as a logging and telemetry-integrity problem, not just a versioning problem: remove trust in raw parser error text.
High	SOC / Detection Engineering	Review applications, dashboards, and detection content that surface unsanitized parser errors from text-protocol handling.
Critical	Platform / Application Owners	Hunt for suspicious multi-line or attacker-shaped parser errors in existing logs while patching progresses.
High	SOC / Detection Engineering	Prioritize externally reachable Go services and shared middleware first, especially where logs inform security operations.

14. References

9. Microsoft Security Response Center, `CVE-2026-42507`
10. Go vulnerability database entry `GO-2026-5039`
11. OSV record `GO-2026-5039`
12. Go issue tracker `#79346`
13. Go release history for `1.25.11`
14. Go release history for `1.26.4`
15. NVD entry for `CVE-2026-42507`
16. Debian Security Tracker entry for `CVE-2026-42507`